

LAPORAN PRAKTIKUM
PEMOGRAMAN BERORIENTASI OBJEK



Disusun Oleh:

Gina Ramadhani

NIM: 2511533014

Kelas: B

Dosen Pengampu: Nurfiah, S.ST, M.Kom

Asisten Praktikum: Siti Fadhilah Rahmi

DAPERTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan praktikum dengan tema “Relasi Antar Objek (Class Diagram & Asosiasi)” ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk dokumentasi dan pertanggungjawaban dalam kegiatan praktikum Pemrograman Berorientasi Objek.

Dalam praktikum ini, dibahas mengenai hubungan antar objek melalui class diagram dan asosiasi. Materi tersebut penting untuk memahami bagaimana suatu class dapat saling berhubungan dan berinteraksi dalam membangun sebuah program berorientasi objek. Penulis berharap laporan ini dapat membantu dalam memahami konsep relasi antar objek serta penerapannya dalam pemrograman Java.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan untuk penyempurnaan laporan di masa yang akan datang. Semoga laporan ini dapat memberikan manfaat bagi penulis maupun pembaca.

Padang, September 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN.....	2
A. Langkah Kerja Praktikum	2
B. Tugas/Latihan Praktikum	5
BAB III PENUTUP.....	10
3.1 Kesimpulan.....	10
DAFTAR PUSTAKA.....	11

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pemrograman Berorientasi Objek (PBO) merupakan salah satu konsep pemrograman yang menggunakan objek dan class sebagai dasar dalam membangun sebuah program. Dalam penerapannya, sebuah program tidak hanya terdiri dari satu class, tetapi dapat memiliki beberapa class yang saling berhubungan dan berinteraksi. Oleh karena itu, pemahaman mengenai relasi antar objek menjadi salah satu bagian penting dalam pemrograman berorientasi objek.

Class diagram digunakan untuk menggambarkan struktur class serta hubungan antar class dalam suatu sistem. Salah satu bentuk hubungan tersebut adalah asosiasi, yaitu hubungan yang menunjukkan bahwa suatu objek dapat berinteraksi atau memiliki keterkaitan dengan objek lainnya. Dengan mempelajari class diagram dan asosiasi, mahasiswa dapat memahami bagaimana hubungan antar objek dirancang sebelum diterapkan ke dalam kode program. Hal ini dapat membantu dalam membuat program yang lebih terstruktur, mudah dipahami, dan sesuai dengan konsep Pemrograman Berorientasi Objek.

1.2 Tujuan Praktikum

1. Membaca dan menerjemahkan Class Diagram ke dalam kode program Java.
2. Memahami dan mengimplementasikan relasi Asosiasi (one to many).
3. Manajemen koleksi objek di dalam objek lain menggunakan tipe data bentukan dan struktur data dinamis (ArrayList).

1.3 Manfaat Praktikum

1. Memahami konsep relasi antar objek dan class dalam Pemrograman Berorientasi Objek.
2. Mengetahui cara menggambarkan hubungan antar class menggunakan class diagram.
3. Memahami penerapan asosiasi sebagai salah satu bentuk hubungan antar objek.

BAB II

PEMBAHASAN

A. Langkah Kerja Praktikum

a) Langkah 1: Membuat Class Diagram

1. Buatlah Sebuah Class Bernama Transaksi

```
1 package pekan2;  
2  
3 public class Transaksi {
```

2. Buat atribut dan constructor pada class Transaksi Class Transaksi memiliki 3 atribut yaitu idTransaksi, jenis dan nominal. Jenis transaksi ada 2 buah yaitu Debit digunakan untuk penarikan dan Kredit digunakan untuk penyetoran.

```
1 package pekan2;  
2  
3 public class Transaksi {  
4     String idTransaksi;  
5     String jenis;  
6     double nominal;  
7  
8     //konstruktor  
9     public Transaksi(String id, String jenis, double nominal) {  
10         this.idTransaksi = id;  
11         this.jenis = jenis;  
12         this.nominal = nominal;  
13     }
```

1) String idTransaksi;

→ Mendeklarasikan atribut idTransaksi bertipe String untuk menyimpan identitas atau kode transaksi.

2) String jenis;

→ Mendeklarasikan atribut jenis bertipe String untuk menyimpan jenis transaksi, misalnya "Setor" atau "Tarik".

3) double nominal;

→ Mendeklarasikan atribut nominal bertipe double untuk menyimpan jumlah uang dalam transaksi.

4) public Transaksi(String id, String jenis, double nominal) {

→ Merupakan constructor dari class Transaksi. Constructor digunakan untuk memberikan nilai awal pada atribut ketika objek Transaksi dibuat.

→ String id, String jenis, dan double nominal merupakan parameter/argumen yang digunakan untuk menerima nilai dari objek yang dibuat.

5) `this.idTransaksi = id;`

→ Mengisi atribut idTransaksi dengan nilai dari parameter id. Kata this menunjukkan bahwa idTransaksi merupakan atribut milik objek saat ini.

6) `this.jenis = jenis;`

→ Mengisi atribut jenis dengan nilai yang diberikan melalui parameter jenis.

7) `this.nominal = nominal;`

→ Mengisi atribut nominal dengan nilai yang diberikan melalui parameter nominal.

3. Membuat method

Kita akan membuat method `cetakDetail()` yang nantinya akan digunakan untuk menampilkan 1 baris detail transaksi.

```
1 package pekan2;
2
3 public class Transaksi {
4     String idTransaksi;
5     String jenis;
6     double nominal;
7
8     //konstruktor
9     public Transaksi(String id, String jenis, double nominal) {
10         this.idTransaksi = id;
11         this.jenis = jenis;
12         this.nominal = nominal;
13     }
14     public void cetakDetail() {
15         System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
16     }
17 }
```

Method `cetakDetail()` merupakan method yang digunakan untuk menampilkan informasi dari objek Transaksi ke layar. Kata `public` menunjukkan bahwa method dapat diakses dari class lain, sedangkan `void` berarti method tidak mengembalikan nilai. Perintah `System.out.println()` digunakan untuk mencetak informasi berupa idTransaksi, jenis, dan nominal ke console. Tanda `+` digunakan untuk menggabungkan teks dengan nilai dari setiap atribut sehingga menghasilkan tampilan detail transaksi dalam satu baris.

b) Langkah 2: Membuat Relasi pada Rekening.java

Buka kembali file Rekening.java dari Praktikum sebelumnya. Kita akan menambahkan atribut baru yang merepresentasikan relasi has-a yaitu satu rekening memiliki banyak transaksi.

```
4 import java.util.ArrayList;
5
6 public class Rekening {
7     String nomorRekening;
8     String namaPemilik;
9     double saldo;
10
11     //Implementasi Asosiasi (1-to-many)
12     ArrayList<Transaksi> riwayatTransaksi;
13
14     public Rekening(String nomor, String nama, double saldoAwal) {
15         nomorRekening = nomor;
16         namaPemilik = nama;
17         saldo = saldoAwal;
18
19         //Wajib menginisialisasi ArrayList di dalam konstruktor agar tidak NullPointerException
20         this.riwayatTransaksi = new ArrayList<>();
21
22         System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
23     }
24
25     //... (pertahankan method getNomorRekening() dan cekInformasi() yang sudah ada
```

Class Rekening digunakan untuk merepresentasikan sebuah rekening yang memiliki nomor rekening, nama pemilik, dan saldo. import java.util.ArrayList digunakan agar program dapat menggunakan ArrayList, sedangkan ArrayList<Transaksi> riwayatTransaksi merupakan penerapan asosiasi 1-to-many, yaitu satu objek Rekening dapat memiliki banyak objek Transaksi sebagai riwayat transaksi. Constructor Rekening(String nomor, String nama, double saldoAwal) digunakan untuk memberikan nilai awal pada atribut nomorRekening, namaPemilik, dan saldo melalui parameter yang diterima. Baris this.riwayatTransaksi = new ArrayList<>(); digunakan untuk menginisialisasi daftar riwayat transaksi agar dapat digunakan dan tidak mengalami NullPointerException. Selanjutnya, System.out.println() digunakan untuk menampilkan informasi bahwa rekening atas nama pemilik berhasil dibuat beserta saldo awalnya.

c) Langkah 3 : Mencatat Riwayat Transaksi

Perbaiki method setorTunai dan tarikTunai di class Rekening.java agar secara otomatis menginstansiasi objek Transaksi dan menyimpannya ke dalam riwayatTransaksi saat transaksi berhasil.

```

public void setorTunai(double nominal) {
    if (nominal > 0) {
        saldo += nominal;
        //Merekam riwayat (Pembuatan objek Transaksi di dalam method)
        String idTrx = "TRX-S-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
    } else {
        System.out.println("Gagal: Nominal setor harus lebih dari 0!");
    }
}

```

Kode di atas merupakan class Rekening yang digunakan untuk membuat objek rekening dan menerapkan asosiasi 1-to-many dengan class Transaksi. `ArrayList<Transaksi> riwayatTransaksi` digunakan untuk menyimpan banyak transaksi yang dimiliki oleh satu rekening. Constructor `Rekening(String nomor, String nama, double saldoAwal)` berfungsi memberikan nilai awal pada nomor rekening, nama pemilik, dan saldo melalui nomor, nama, dan saldoAwal. Kemudian `this.riwayatTransaksi = new ArrayList<>();` digunakan untuk membuat daftar kosong yang akan menampung riwayat transaksi sehingga dapat digunakan tanpa mengalami `NullPointerException`. Terakhir, `System.out.println()` digunakan untuk menampilkan informasi bahwa rekening berhasil dibuat beserta nama pemilik dan saldo awalnya.

B. Tugas/Latihan Praktikum

1. Integrasi Rekam Jejak Penarikan (30%)

- Buka kembali method `tarikTunai(double nominal)` di dalam file `Rekening.java`.
- Tambahkan logika perekaman objek Transaksi seperti pada contoh `setorTunai`. Pastikan jenis transaksinya dicatat sebagai "Debit" dan kodenya menggunakan prefiks "TRX-T". Transaksi hanya boleh dicatat jika penarikan berhasil (saldo mencukupi).

```

40 public void tarikTunai(double nominal) {
41     if (nominal < 10000) {
42         System.out.println("Transaksi Gagal: Minimal nominal penarikan 10.000.");
43     } else if (nominal > saldo) {
44         System.out.println("Transaksi Gagal: Saldo tidak mencukupi. "
45             + "Saldo Anda: Rp" + saldo);
46     } else {
47         saldo -= nominal;
48
49         //Merekam riwayat transaksi penarikan
50         String idTrx = "TRX-T-" + System.currentTimeMillis();
51         Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
52         riwayatTransaksi.add(trxBaru);
53
54         System.out.println("Tarik tunai Rp" + nominal
55             + " berhasil. Saldo saat ini: Rp" + saldo);
56     }
57 }

```

Kode tersebut merupakan method tarikTunai(double nominal) yang digunakan untuk melakukan penarikan uang sekaligus mencatat transaksi penarikan. Pada bagian if (nominal < 10000), kode berhasil ditambahkan untuk memeriksa batas minimal penarikan, sedangkan else if (nominal > saldo) digunakan untuk mengecek apakah saldo mencukupi. Jika penarikan berhasil, kode berhasil ditambahkan untuk mengurangi saldo dan merekam transaksi, yaitu membuat ID dengan awalan "TRX-T-", membuat objek Transaksi dengan jenis "Debit", kemudian memasukkannya ke dalam riwayatTransaksi. Dengan demikian, setiap penarikan yang berhasil akan tercatat sebagai riwayat transaksi, sedangkan penarikan yang gagal tidak akan dicatat.

Maka sekarang output program akan seperti ini

```

=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu:
1
Masukkan No Rekening: 35636
Masukkan Nama Pemilik: gina
Masukkan Saldo Awal: 380000
Rekening atas nama gina berhasil dibuat dengan saldo Rp380000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu:
2
Masukkan nominal setor: 2000
Setor tunai Rp2000.0 berhasil. Saldo saat ini: Rp382000.0

```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu:
3
Masukkan nominal tarik tunai: 200000
Tarik tunai Rp200000.0 berhasil. Saldo saat ini: Rp182000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu:
4
--- INFO REKENING ---
n0. Rekening : 35636
Nama Pemilik : gina
Saldo Akhir : Rp182000.0
-----

```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu:
6
--- RIWAYAT TRANSAKSI ---
ID: TRX-S-1789912728502 | Jenis: Kredit | Nominal: Rp2000.0
ID: TRX-T-1789912735068 | Jenis: Debit | Nominal: Rp200000.0
-----

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu:

```

2. Fitur Cetak Mutasi Rekening (70%)

- Buat sebuah method baru di dalam class Rekening.java dengan nama cetakMutasi().
- Di dalam method tersebut, gunakan struktur perulangan for-each untuk menelusuri seluruh data di dalam koleksi riwayatTransaksi. Panggil method cetakDetail() milik setiap objek Transaksi. Jika riwayat kosong, cetak pesan: "Belum ada transaksi pada rekening ini".
- Buka file Main.java, tambahkan menu baru: "6. Cetak Mutasi (Riwayat)". Panggil method cetakMutasi() milik akunAktif saat pengguna memilih menu tersebut.

Tambahkan kode dibawah ini di class Rekening

```

61 public void cetakMutasi() {
62     if (riwayatTransaksi.isEmpty()) {
63         System.out.println("Belum ada transaksi pada rekening ini.");
64     } else {
65         System.out.println("--- RIWAYAT TRANSAKSI ---");
66         for (Transaksi transaksi : riwayatTransaksi) {
67             transaksi.cetakDetail();
68         }
69         System.out.println("-----");
70     }
71 }

```

Kemudian tambahkan kode di bawah ini di class Main

```

case 6:
    akunAktif.cetakMutasi();
    break;

```

Maka Output program seperti dibawah ini:

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu:
6
--- RIWAYAT TRANSAKSI ---
ID: TRX-S-1789912728502 | Jenis: Kredit | Nominal: Rp2000.0
ID: TRX-T-1789912735068 | Jenis: Debit | Nominal: Rp200000.0
-----

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu:

```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa relasi antar objek merupakan bagian penting dalam Pemrograman Berorientasi Objek karena memungkinkan beberapa class untuk saling berhubungan dan berinteraksi. Class diagram dapat digunakan untuk menggambarkan struktur class beserta hubungan yang terdapat di dalam suatu sistem, sedangkan asosiasi digunakan untuk menunjukkan keterkaitan antara satu objek dengan objek lainnya. Dengan memahami konsep tersebut, perancangan program dapat dilakukan dengan lebih terstruktur sehingga memudahkan proses implementasi dan pengembangan program menggunakan Java.

DAFTAR PUSTAKA

Nurfiah. (2026). *Modul praktikum 2 pemrograman berorientasi objek–Java*.
Fakultas Teknologi Informasi, Universitas Andalas.